Deliciousthings

Espace de travail collaboratif
Architecture et modèles pour le MVP

Projet-first, demande-first et gestion artisan - version d'exécution

21 août 2026

Décision structurante
Le projet est un espace privé de travail. Une publication publique n'est jamais le projet lui-même : elle prend la forme d'un PortfolioItem créé par l'artisan. Pour le MVP, seules les photos dont l'artisan maîtrise les droits de publication sont utilisables dans son portfolio.

1. Objectif du MVP

Le MVP doit permettre à Deliciousthings de couvrir trois parcours avec une architecture commune, sans multiplier les rôles globaux ni construire trop tôt un système complexe de consentements et de droits éditoriaux.

Parcours	Entrée	Résultat attendu
Projet-first	Un utilisateur crée d'abord un projet personnel.	Il structure son besoin, puis crée une demande à partir du projet et invite un artisan après accord.
Demande-first	Un utilisateur contacte d'abord un artisan.	Une demande acceptée peut donner naissance à un projet collaboratif.
Artisan-first	Un artisan a déjà un client, éventuellement hors Deliciousthings.	Il crée et gère un projet professionnel puis invite éventuellement le client.

Principe de rôle
User est l'identité. Artisan est un profil professionnel optionnel. 'Client' est principalement un rôle contextuel dans ProjectParticipant, pas un type d'utilisateur exclusif.

2. Périmètre fonctionnel à livrer

- Création d'un compte et authentification.
- Création d'un projet privé par tout utilisateur authentifié.
- Projet utilisable comme idée, préparation ou projet actif grâce à un statut simple.
- Invitation d'un participant : client, artisan ou collaborateur.
- Création d'une demande autonome ou préremplie depuis un projet existant.
- Création d'un projet à partir d'une demande acceptée.
- Collaboration minimale dans le projet : photos/fichiers, commentaires, tâches et jalons.
- Messagerie contextuelle en réutilisant le modèle Message existant.
- Profil Artisan et portfolio public indépendant du projet privé.
- Création manuelle d'un PortfolioItem avec les médias de l'artisan ; source_project_id peut conserver la provenance sans publier le projet.

Hors MVP

- Version publique complète d'un projet avec sélection collaborative des sections.
- Workflow fin de consentement photo par photo entre client et artisan.
- Journal personnel automatique du client généré à la fin d'un projet.
- Permissions personnalisées par participant ou par artefact.
- Historique de versions, audit détaillé et workflow d'approbation éditoriale.
- Conversation générique multi-canaux avec pièces jointes et réactions avancées.

3. Modèles existants : rôle dans le MVP

Modèle actuel	Décision MVP	Responsabilité
User	Conserver	Identité, authentification, propriétaire de projets, auteur de contenus, participant.
Artisan	Conserver	Profil professionnel rattaché à un User. Porte les informations publiques du professionnel.
Address	Conserver	Adresse utilisateur/artisan ; peut aussi être référencée par un projet si nécessaire.

Message	Étendre légèrement	Message direct ou contextuel à une demande/un projet. Pas besoin d'un modèle Conversation au MVP.
Service	Conserver	Catalogue de services d'un Artisan. Le lien avec une demande peut attendre si non indispensable au premier flux.

4. Nouveaux modèles indispensables

4.1 Project

Le coeur de l'espace de travail. Un Project est toujours privé dans le MVP. Il peut exister sans demande et sans client inscrit.

Champ	Type suggéré	Notes
id	bigint	PK
owner_id	FK users	Créateur/propriétaire du projet.
source_inquiry_id	FK inquiries nullable	Demande d'origine, si le projet a été créé depuis une demande.
title	string	Nom du projet.
status	enum/string	idea, planning, active, completed, cancelled.
event_date	datetime nullable	Date de l'événement.
budget	decimal nullable	Valeur MVP ; une plage min/max peut venir plus tard.
servings	unsigned int nullable	Nombre de portions.
description	text nullable	Besoin libre.
details	json nullable	Styles, allergies, saveurs, retrait, installation, etc. tant que le schéma évolue.
address_id	FK addresses nullable	Lieu/livraison si déjà pertinent.
created_at / updated_at	timestamps	Standard Laravel.

Project belongsTo User as owner
Project hasMany ProjectParticipant
Project hasMany ProjectMedia / ProjectComment / ProjectTask / ProjectMilestone
Project hasMany Message (optional context)
Project hasMany PortfolioItem through source_project_id (provenance only)

4.2 ProjectParticipant

Ce pivot explicite remplace le besoin d'un modèle Client global et permet aussi d'inviter quelqu'un qui n'a pas encore de compte.

Champ	Type suggéré	Notes
project_id	FK projects	Projet concerné.
user_id	FK users nullable	Null tant que l'invité n'a pas rejoint Deliciousthings.

invited_email	string nullable	Invitation externe.
display_name	string nullable	Nom du client externe si utile.
role	enum/string	owner, client, artisan, collaborator.
status	enum/string	pending, active, declined, removed.
invited_by	FK users	Auteur de l'invitation.
joined_at	datetime nullable	Activation de la participation.

Important
Le rôle 'artisan' dans un projet n'accorde aucun droit de publication du projet. Le contrôle du projet reste lié au propriétaire ; le portfolio appartient au profil Artisan.

4.3 Inquiry (la demande)

Utiliser Inquiry plutôt que Request évite une collision sémantique et technique avec Illuminate\Http\Request. La demande est un snapshot commercial : lorsqu'elle est créée depuis un projet, les données utiles sont copiées afin que la demande envoyée ne change pas silencieusement si le projet évolue.

Champ	Type suggéré	Notes
creator_id	FK users	Utilisateur à l'origine de la demande.
recipient_artisan_id	FK artisans nullable	Artisan Deliciousthings ciblé.
recipient_email	string nullable	Pour une invitation externe au MVP si nécessaire.
source_project_id	FK projects nullable	Projet utilisé pour préremplir la demande.
resulting_project_id	FK projects nullable	Projet créé/lié après accord.
status	enum/string	draft, sent, accepted, declined, cancelled.
title	string	Objet.
description	text nullable	Besoin communiqué.
event_date	datetime nullable	Snapshot.
budget	decimal nullable	Snapshot.
servings	unsigned int nullable	Snapshot.
details	json nullable	Autres données importées du projet.
sent_at / responded_at	datetime nullable	Étapes principales.

4.4 Artefacts du projet : modèles simples, pas de méga-table JSON

Pour le MVP, quatre petits modèles spécialisés sont plus simples à valider et exploiter qu'un Artifact générique qui porterait des structures très différentes.

Modèle	Champs minimum	Usage
ProjectMedia	project_id, created_by, path, type, caption nullable	Photos et fichiers du projet.
ProjectComment	project_id, user_id, body, project_media_id nullable	Commentaires généraux ou sur une photo.

ProjectTask	project_id, title, description nullable, completed_at nullable, assigned_to nullable	To-do list.
ProjectMilestone	project_id, title, starts_at nullable, position, notes nullable	Déroulement / étapes importantes.

4.5 PortfolioItem et PortfolioMedia

Le portfolio ne publie pas le Project. Il crée une représentation professionnelle séparée, appartenant à Artisan.

Modèle	Champs minimum	Règle MVP
PortfolioItem	artisan_id, source_project_id nullable, title, description nullable, status, published_at nullable	Seul l'artisan propriétaire du profil peut créer/modifier/publier.
PortfolioMedia	portfolio_item_id, source_project_media_id nullable, path, caption nullable, position	Au MVP : n'autoriser automatiquement que les médias ajoutés par l'artisan lui-même ou importés explicitement par lui.

5. Carte relationnelle MVP

```
User 1 ----- 0..1 Artisan
  |\
  | \----< Inquiry >---- 0..1 Artisan (recipient)
  |
  +------< Project (owner)
           |
           +------< ProjectParticipant >------ 0..1 User
           +------< ProjectMedia
           +------< ProjectComment
           +------< ProjectTask
           +------< ProjectMilestone
           +------< Message
           |
           +------< PortfolioItem (source only) >------ Artisan
                                  |
                                  +------< PortfolioMedia
```

Relations Eloquent prioritaires

Depuis	Relation	Cible
User	hasOne	Artisan
User	hasMany	Project as owner
Project	belongsTo	User as owner
Project	hasMany	ProjectParticipant
ProjectParticipant	belongsTo nullable	User
Inquiry	belongsTo	User as creator
Inquiry	belongsTo nullable	Artisan as recipient
Inquiry	belongsTo nullable	Project as source/result
Artisan	hasMany	PortfolioItem

PortfolioItem	belongsTo nullable	Project as source
PortfolioItem	hasMany	PortfolioMedia

6. Permissions à implémenter avec Policies

Action	Owner du projet	Participant client	Participant artisan	Artisan profile owner
Voir le projet	Oui	Oui si active	Oui si active	-
Modifier données structurantes	Oui	Limité / non au MVP	Limité / non au MVP	-
Ajouter média/commentaire/tâche	Oui	Oui	Oui	-
Inviter/retirer participant	Oui	Non	Non	-
Créer une demande depuis projet	Oui	Éventuellement si owner uniquement au MVP	Non par défaut	-
Publier le projet lui-même	Non : fonctionnalité absente	Non	Non	-
Créer PortfolioItem lié au projet	-	Non	Oui si possède Artisan + participe	Oui
Publier PortfolioItem	-	Non	Seulement sur son Artisan	Oui

Règle de sécurité
La participation à un Project donne un droit de collaboration, jamais un droit implicite de publication. Une source_project_id est une provenance, pas une autorisation.

7. Adaptation du modèle Message existant

Pour ne pas reconstruire une messagerie complète, le MVP peut conserver Message et lui ajouter des contextes optionnels.

Champ conseillé	But
sender_id	Auteur du message.
recipient_id nullable	Destinataire direct si nécessaire.
project_id nullable	Message associé à un projet.
inquiry_id nullable	Message associé à une demande.
body	Contenu.
read_at nullable	Lecture simple.

Si la messagerie de projet devient un véritable chat de groupe, introduire Conversation et ConversationParticipant plus tard. Le MVP n'en a pas besoin si les commentaires projet couvrent déjà une grande partie des échanges.

8. Flux applicatifs à implémenter

8.1 Projet-first

1. User crée Project.

2. Il ajoute ses informations, médias et tâches.

3. Il choisit 'Créer une demande'.

4. Le serveur copie les données pertinentes de Project vers Inquiry.

5. Inquiry est envoyée à Artisan.

6. Après acceptation, Artisan est ajouté à ProjectParticipant avec role=artisan, status=active.

7. La collaboration continue dans le Project.

8.2 Demande-first

1. User crée Inquiry sans project_id.

2. Artisan accepte.

3. Action 'Créer le projet' crée Project et copie les champs de la demande.

4. Inquiry.resulting_project_id est renseigné.

5. Créateur et artisan deviennent ProjectParticipant actifs.

8.3 Artisan-first

1. User possédant Artisan crée Project.

2. Il reste owner du projet et peut enregistrer un client externe via ProjectParticipant (user_id null, display_name/email).

3. Le projet fonctionne même si le client ne rejoint jamais la plateforme.

4. Si le client accepte l'invitation plus tard, son User est rattaché au participant existant.

8.4 Fin de projet et portfolio

1. Project passe à completed.

2. L'artisan peut créer un PortfolioItem avec source_project_id.

3. Il sélectionne uniquement ses propres médias ou téléverse de nouvelles photos dans PortfolioMedia.

4. Il édite titre/description et publie le PortfolioItem.

5. Les médias appartenant au client nécessitant une autorisation restent hors MVP.

9. Écrans minimum nécessaires

Écran	Responsabilité
Accueil authentifié	Fallback sans projet + liste récente + CTA Nouveau projet / Nouvelle demande / Profil professionnel.
Créer / éditer projet	Champs essentiels ; détails avancés dans JSON côté backend mais formulaires progressifs.
Détail projet	Résumé, participants, médias, commentaires, tâches, jalons.
Inviter un participant	Email + rôle contextuel.
Créer une demande	Autonome ou préremplie depuis Project.
Détail demande	Statut, contenu envoyé, réponse/acceptation.
Dashboard artisan	Projets professionnels, demandes reçues, accès portfolio.
Éditeur PortfolioItem	Médias, titre, description, brouillon/publication.
Profil artisan public	PortfolioItem publiés + Services existants.

10. Ordre d’implémentation Laravel conseillé

1. Créer migrations + modèles : Project, ProjectParticipant, Inquiry.

2. Créer ProjectPolicy et InquiryPolicy avant l'interface collaborative.

3. Implémenter création/édition Project et invitation participant.

4. Ajouter ProjectMedia, ProjectComment, ProjectTask, ProjectMilestone progressivement.

5. Étendre Message uniquement si un besoin de messagerie distinct des commentaires existe immédiatement.

6. Implémenter les transformations Project -> Inquiry et Inquiry -> Project dans des Actions/Services dédiés, pas dans les controllers.

7. Créer PortfolioItem + PortfolioMedia et une PortfolioItemPolicy.

8. Brancher les écrans Inertia/Vue sur les Resources Laravel avec payloads ciblés.

9. Ajouter tests feature des permissions et des transformations avant d'étendre le modèle de consentement.

Actions/Services utiles

```
CreateInquiryFromProject
CreateProjectFromInquiry
InviteProjectParticipant
AcceptProjectInvitation
CreatePortfolioItemFromProject
```

Ces classes encapsulent les copies de données et les transitions de statut ; les Controllers restent minces.

11. Enums minimales

Enum	Valeurs MVP
ProjectStatus	idea, planning, active, completed, cancelled
ProjectParticipantRole	owner, client, artisan, collaborator
ProjectParticipantStatus	pending, active, declined, removed
InquiryStatus	draft, sent, accepted, declined, cancelled
PortfolioItemStatus	draft, published

12. Tests indispensables avant mise en ligne

- Un utilisateur ne peut pas lire/modifier un projet auquel il ne participe pas.
- Seul l'owner peut inviter ou retirer des participants.
- Un artisan invité peut collaborer mais ne peut pas publier le Project.
- Une Inquiry créée depuis un Project garde son snapshot si le Project change ensuite.
- Une Inquiry acceptée peut produire un seul Project résultant (idempotence).
- Un artisan peut créer un Project avec un participant client non inscrit.
- Seul le propriétaire d'un Artisan peut publier ses PortfolioItem.
- Un PortfolioItem lié à un Project n'expose jamais automatiquement les ProjectMedia.
- Les routes publiques d'un portfolio ne sérialisent aucune donnée privée du Project source.

13. Schéma MVP final recommandé

En tenant compte des modèles déjà présents, la version suffisante pour exécuter l'espace de travail est la suivante :

Conserver	Ajouter maintenant	Reporter

User Artisan Address Message Service	Project ProjectParticipant Inquiry ProjectMedia ProjectComment ProjectTask ProjectMilestone PortfolioItem PortfolioMedia	Client global Conversation PublicationConsent PublicProject/ProjectShare JournalEntry Permissions par artefact Versioning/audit avancé

Critère de réussite

Avec ce schéma, Deliciousthings peut déjà gérer un projet personnel, une demande, une collaboration client-artisan, un client externe et un portfolio professionnel, sans faire du projet privé une ressource publique.

14. Décisions à ne pas figer trop tôt

- Ne pas créer Client comme modèle d'identité tant qu'aucun attribut métier propre et durable ne le justifie.
- Ne pas rendre Project polymorphe entre 'projet client' et 'projet artisan' : le contexte vient de owner + participants.
- Ne pas introduire un Artifact générique tant que les quatre types de contenu n'ont pas montré un vrai besoin d'unification.
- Ne pas créer un système de consentement complet tant que l'import de médias clients vers le portfolio n'est pas une fonctionnalité MVP.
- Ne pas utiliser la visibilité publique comme état du Project. La publication professionnelle reste PortfolioItem.